Our Capstone Project — The Full Story

Multimodal Myopia & Eye Disease Prediction from Fundus Photos

Everything we did, step by step: what, why, what the other options were, and what we chose.

Bhanu Shankar Prajwal P · Final-year CSE, Dayananda Sagar University

Guide: Dr. Revathi V · Teammate: Tejas C E

October 2026

How to read this
Chapters 0–17 tell the story in order. Then come the things we chose NOT to do, the limitations, the tech stack, and a one-page summary at the end. Section numbers in brackets match the master project document.

Chapter 0: The starting point

What we did

- Original topic: myopia prediction. After the first review the guide asked us to cover **myopia and other eye diseases**.
- Goal: predict 8 conditions — Normal, Diabetes, Glaucoma, Cataract, AMD, Hypertension, Myopia, Others — from **patient details (age, sex)** and **fundus photos** (photos of the back of the eye).
- Dataset: **ODIR-5K** (full_df.csv) — 6,392 photos from 3,358 patients (left + right eye). A well-known public dataset from Chinese hospitals, used in many research papers.

Why we did it

- It has patient details and photos together, and covers all 8 conditions.
- Plan: start simple, get a working result, then improve step by step. We did not build the full photos + clinical system on day one, because if it failed we would not know which part was wrong.

Other options we had
• Datasets with only myopia or only diabetic eye disease.

What we chose, and why
• ODIR-5K, because the guide wanted many diseases and it is a standard benchmark dataset.

Chapter 1: Looking at the data (Section 4)

What we did

- Checked size, columns, count of each disease, age range, missing values and duplicates.

Why we did it

- You must know the data before building anything, or you train on mistakes without knowing.

What we found

- 6,392 rows and 3,358 patients — as expected.
- **Myopia is rare** (about 4–5% of rows) — this is **class imbalance**.
- **15.7% of rows have 2–3 diseases at once** — not perfectly one disease per photo.
- **28 rows have age = 1** — strange, because the next lowest age is 14.
- No missing values, no duplicate rows.

Chapter 2: Cleaning the data and choosing features (Section 5)

What we did

- **Age = 1:** marked as missing, later filled with the training median (59).
- **Feature selection:** sorted all 19 columns. IDs/filenames = useless. Diagnostic keywords, disease flags, labels, target = **they contain the answer**. Only **age and sex** are fair inputs.
- **Two label columns disagree** on 27% of rows (myopia: 306 vs 232). We used the **labels** column — it matches the official count (232) and is recorded per photo (per eye).

Why we did it

- Age 1 is almost surely a placeholder for 'not recorded'. 89% of those rows were myopia, so keeping it would teach the model 'age 1 = myopia' — a fake shortcut (**leakage**).
- If the model sees the doctor's diagnosis as input, it scores near 100% — but that is fake (**target leakage**). Examiners always check for this.
- Only 2 clinical features are left — a limitation of the dataset (no refraction, eye pressure or family history), not our mistake.

Other options we had
• Delete the age-1 rows (loses rare myopia cases).
• Add a 'was age missing' column (89% linked to myopia — cheating).
• Use all columns for high accuracy.
• Automatic feature selection (SelectKBest, RFE).

What we chose, and why
• Mark age 1 as missing — keeps the data, removes the fake signal.
• Refused to use leaking columns — an honest result is worth more than a high number.
• No SelectKBest/RFE: with only 2 legal features there is nothing to select. Our feature selection was done by reasoning about which columns leak the answer.

Chapter 3: Preparing the data and the patient-level split (Section 7)

What we did

- **Patient-level split:** all photos of one patient go to the same side (train or test). Checked in code: **zero patients** in both.
- **Stratified:** similar myopia share on both sides (3.66% train vs 3.50% test).
- **Sex → 0/1** (one-hot encoding). **Age → standardised** (mean 0, spread 1).
- The median filler and scaler were fitted on **training data only**, then applied to test.

Why we did it

- Left and right eyes of one person look very similar (same person, camera, day). If one is in train and the other in test, the model has already 'seen' that person and the score is unfairly high.
- Stratifying avoids a test set with almost no myopia by bad luck.
- Models need numbers, not words. Scaling stops one input's size from overpowering others (helps LR and SVM).
- Fitting on all data would leak test information into training.

Other options we had
• Random split by photo — easier, higher numbers, but inflated. Many published papers do this.
• Min-max normalisation instead of standardisation.

What we chose, and why
• Patient-level split — the honest way; examiners respect it.
• Standardisation — the normal choice for LR/SVM, and it meets the 'normalise/standardise' requirement.

Chapter 4: First classifiers using age + sex only (Experiment 1, Section 8)

What we did

- Predicted Myopia vs Non-Myopia with **Logistic Regression, Random Forest and SVM**.
- Handled imbalance with **class weights** (a myopia mistake costs about 13.7× more).

Why we did it

- They are the classic baselines: LR is simple and explainable, RF handles non-linear patterns, SVM is a strong classical method.
- Conclusion: age + sex alone cannot predict myopia well. That told us **photos are needed** — this result is why the image part exists.

Results (1,284 test rows)

Model	Accuracy	Macro-F1	Myopia recall
Random Forest	74.8%	0.464	28.9%
Logistic Regression	lower	lower	62.2%
SVM	lower	lower	55.6%

Myopia precision was only 4–5% for all three. **Why accuracy is not enough:** a model that always says 'no myopia' gets ~96% accuracy and catches zero cases. So we used **macro-F1** (treats both classes equally) and **recall** (how many real cases we catch).

Other options we had
- SMOTE / oversampling (creates fake rows).
- Undersampling (throws data away).
- Do nothing (model ignores myopia).

What we chose, and why
- Class weights — no fake data and no data thrown away.

Chapter 5: Checking the photo folders (Section 3.1)

What we did

- Counted and checked every photo folder before training anything.
- **datasets/<disease>**: counts match the CSV exactly, but filenames (image0.png…) have **no patient ID** — cannot link to age/sex.
- **preprocessed_images/**: the same 6,392 photos, **named by patient ID** (e.g. 0_left.jpg) — linkable.
- **IMAGES/** (~124,000 photos): **rejected** — mostly exact copies (checked with file hashes), and myopia photos had a **streak mark** normal ones did not (the model could learn 'streak = myopia').
- Some photos had **red and blue swapped** (look blue) — we later added an automatic colour fix.

Why we did it

- The brief said labels might be wrong, so we checked first. A big folder is not automatically good data.

Other options we had
• Use the huge IMAGES folder for 'more data'.

What we chose, and why
• Used preprocessed_images (linkable to patients) — and rejected IMAGES because duplicates and the streak shortcut would give fake results.

Chapter 6: First photo model: Myopia vs Normal (Sections 3.2–3.3)

What we did

- First try in the cloud: pre-trained weights could not download, so it started random and learned nothing (said 'Normal' for all). Not a failure — it proved the code pipeline works; written down honestly.
- Moved training to the laptop GPU (**RTX 4060**).
- Switched **TensorFlow → PyTorch**, and created a **Python 3.11** environment (venv311).
- Model: **MobileNetV2 with transfer learning**.
- Result: **96.6% accuracy, 89.5% macro-F1**; caught 34 of 35 myopia photos (some false alarms).

Why we did it

- **Transfer learning:** the network was already trained on millions of everyday photos (ImageNet), so it already knows edges, shapes and textures. With ~3,000 photos, training from zero would fail.
- TensorFlow no longer supports the GPU on Windows directly (only via WSL2); PyTorch does. Python 3.14 was not yet supported by PyTorch.
- For screening, false alarms are acceptable; missed cases are worse.

Other options we had
• Train in Colab.
• ResNet or EfficientNet instead of MobileNetV2.
• Train from scratch.

What we chose, and why
• Local GPU — faster, no download blocks.
• MobileNetV2 — small and fast, good for a first test (bigger models came later).
• Limitation stated openly: this folder had no patient IDs, so the split was by photo. Fixed in the next steps.

Chapter 7: Combining photos + age/sex for myopia — fusion (Sections 3.4–3.5)

What we did

- This is the **'multimodal'** part of the title.
- **First version:** the photo model gives 64 numbers, age + sex give 8 numbers; joined, and a small layer decides. Used preprocessed_images with a **patient-level split**. Result: 90.5% accuracy, 72.4% macro-F1 — **worse than photo-only**.
- **Improved version:** let the last 5 blocks of the photo network keep learning together with the fusion layer (**fine-tuning**). Final myopia model: **97.4% accuracy, 87.5% macro-F1, 84% recall** on 504 unseen patients. Used in the app's Myopia check.

Why we did it

- Age and sex are weak signals, and in the first version the photo part was frozen (not allowed to adapt).
- Honest point: fusion is close to photo-only but not better, because age/sex add little.

Other options we had
• Keep the photo part frozen and only tune settings (Experiment A).

What we chose, and why
• Fine-tuning (Experiment B) — it helped much more than tuning alone.

Chapter 8: The 8-disease model (Section 3.6)

What we did

- Target: the per-photo label (one disease per photo).
- Split: **StratifiedGroupKFold grouped by patient** — about 4,472 train / 960 validation / 960 test photos; zero overlap.
- **Validation** is used to choose models; the **test set is used once** at the end.
- Results: age + sex only = 10.6% macro-F1 (the 'always Normal' baseline is 7.8%). MobileNetV2 on photos = **49.8% accuracy, 48.1% macro-F1**.

Why we did it

- The guide asked for many diseases.
- Low first score because it was trained on a CPU, with small 224px photos and only part of the network learning.

Other options we had
• Fusion (photos + age/sex) for 8 diseases.

What we chose, and why
• No fusion here — age/sex add almost nothing for 8 diseases, so it would only add complexity.

Chapter 9: First web app (Section 3.7)

What we did

- A **Streamlit** website with a Myopia check page and an 8-disease check page.

Why we did it

- So the models can be used and demonstrated without the command line.

Other options we had
• Flask or React (much more work).
• Gradio (simpler, less flexible).

What we chose, and why
• Streamlit — pure Python and fast to build; good for a prototype.

Chapter 10: Big improvement with EfficientNet-B0 (Section 3.8)

What we did

- Trained on the GPU, with **bigger photos (448px instead of 224px)**, the **whole network learning**, longer training, and **EfficientNet-B0**.
- Result: **71.5% accuracy, 63.5% macro-F1, AUC 0.871** (up from 49.8%).
- Strong: Myopia (AUC 0.996), Cataract, AMD. Weak: Glaucoma and Hypertension (few photos, subtle signs).

Why we did it

- Small signs (tiny bleeds, optic disc shape) get lost when photos are shrunk.
- EfficientNet is more accurate than MobileNetV2 for a similar size.

Other options we had
• Very large models like ResNet-152.

What we chose, and why
• Not used: little data and an 8GB GPU. The benchmark later showed bigger models do not win anyway.

Chapter 11: Grad-CAM heatmaps — explainability (Section 3.9)

What we did

- A heatmap that shows **where in the photo the model looked**.
- AMD → heat on the macula (correct). Myopia → on the pale area near the optic disc (typical sign). Very hazy cataract photos → heat spread out.

Why we did it

- A model can score well by looking at the wrong thing (a border, a camera mark). Examiners trust a model more if it looks at the right place.

Other options we had
• LIME, SHAP, plain saliency maps.

What we chose, and why
• Grad-CAM — the standard for CNNs, and it needs no extra library.

Chapter 12: Ensemble (Section 3.11)

What we did

- Trained a second EfficientNet on 512px photos with **contrast enhancement** (Ben Graham method — makes vessels and lesions stand out).
- Alone it was worse (64.9%), but **averaging it with the first model** was best: **71.8% accuracy, AUC 0.912**.

Why we did it

- Two models make different mistakes; averaging cancels some of them.

What we chose, and why
• Rule we always follow: pick by **validation score, never test score**.

Chapter 13: Testing on a different hospital's data — RFMiD (Section 3.12)

What we did

- **RFMiD:** 3,200 photos from an eye clinic in **India**. Mapped its labels to our 8 classes (Glaucoma and Cataract only approximate).
- Result without retraining: AUC dropped to **0.752**; Myopia dropped most (0.996 → about 0.75).

Why we did it

- All our earlier test photos came from the same hospitals as training. Would it work elsewhere?
- Meaning: the model had partly learned 'how ODIR photos look' (camera, colour), not only the disease. This is **domain shift** — a very common, honest finding in medical AI.

Chapter 14: Training on both datasets (Section 3.13)

What we did

- Added clean RFMiD photos to training (normal, or exactly one of diabetes / AMD / myopia).
- RFMiD AUC rose **0.745 → 0.800**; Myopia **0.71 → 0.91**. ODIR stayed about the same: **72.0% accuracy, 65.5% macro-F1**.

Why we did it

- To fix the domain-shift drop found in Chapter 13.
- Only clean photos, to avoid training on approximate labels.

What we chose, and why
• Honest note: RFMiD is now partly 'seen', so it is no longer a fully unseen-hospital test. A third dataset would be needed for that.

Chapter 15: Batch screening tool (Section 3.14)

What we did

- **Step 1 — photo quality check (rules, not AI):** too dark, over-exposed, not a fundus photo → **Retake**. Blurry photos are **not** rejected (a cataract itself makes a photo hazy — those patients must not be hidden).
- **Step 2 — the AI sorts the rest: Refer** if any disease ≥ 30%; **Likely normal** if Normal ≥ 70%; otherwise **Doctor review**.
- Limits chosen on **validation** with two safety rules: miss at most 5% of diseased eyes; at least 90% of Refer photos must really be diseased.
- Results on 1,600 test photos: **94.6%** of diseased eyes reach a doctor, **4.3%** missed, **91%** of Refer photos really diseased, about a third of healthy eyes cleared.

Why we did it

- In a real eye camp a doctor has hundreds of photos. One-at-a-time checking does not help much — this tool **sorts many photos by urgency**.

Other options we had
• An AI-based photo quality model.

What we chose, and why
• Rule-based quality check — we have no labelled 'bad photo' dataset to train one. Weakness stated openly: it caught only 4 of 30 grader-marked bad photos.

Chapter 16: App redesign

What we did

- Dark navy professional theme; clear navigation (Home, Single photo check, Myopia check, Batch screening, About & results); colour-coded results (red = refer, amber = review, green = normal); heatmaps; downloads.

Why we did it

- We will demo it, so it must be easy to use and easy to explain.

Chapter 17: Comparison with other models (Section 3.15)

The guide asked: *did you compare your model with others, and is it better or worse?* We answered in four ways.

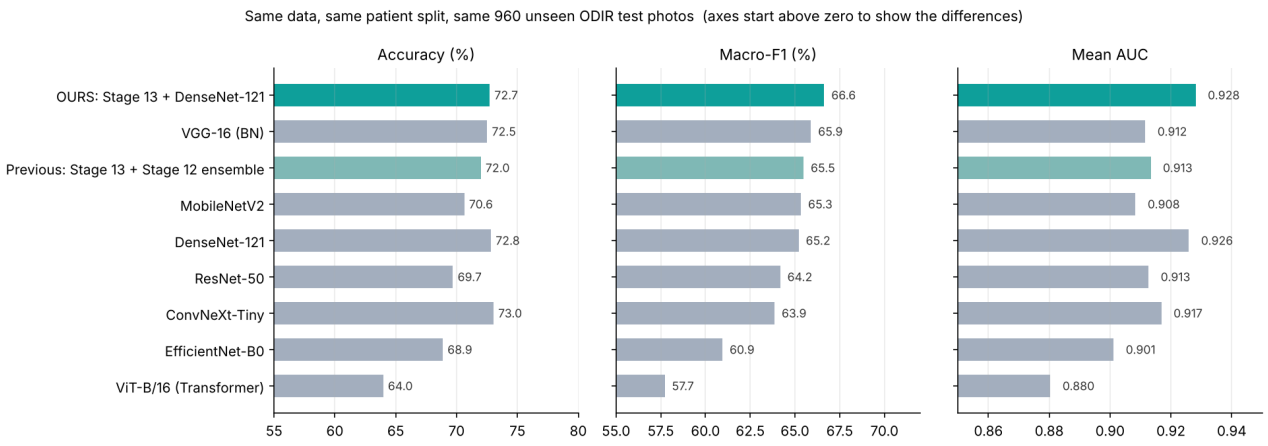
17a. Published papers on ODIR-5K

Study	Their result	Comparable?
ResNet papers (Karthikayan 2024)	76–79% accuracy, F1 0.67–0.71	Partly — split not described, patient leakage possible
Sci. Reports 2025 baselines	EfficientNet-B3 62.1%, DenseNet-201 54.6%	Partly — random split, validation only
ODIR challenge papers	AUC 0.90–0.96, F1 0.88–0.94	No — different task (whole patient, many labels)
Ours	**72.7%, F1 0.666, AUC 0.928**	—

Papers use different splits and scoring, so comparing raw numbers is unfair. That is why we also made our own fair comparison.

17b. Fair benchmark — 7 networks, same data, same split, same settings

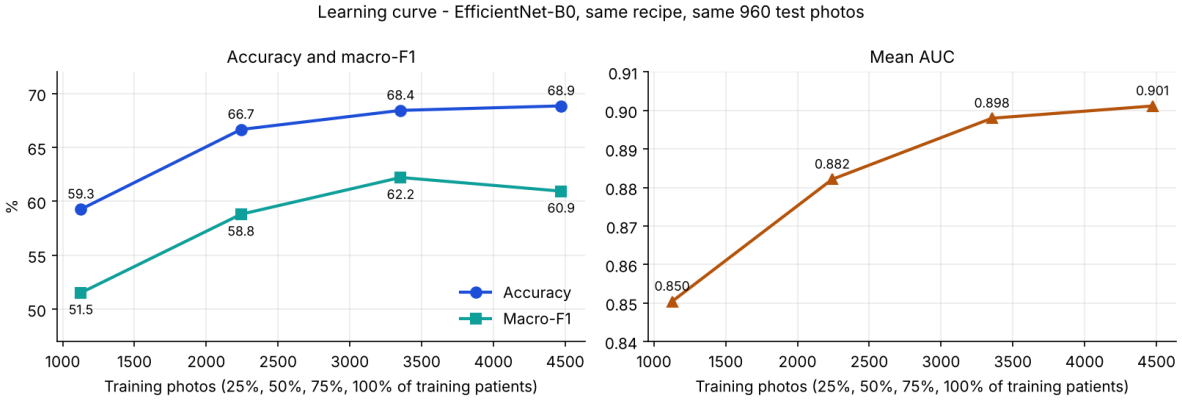
Network	Accuracy	Macro-F1	AUC
Our final (S13 + DenseNet-121)	**72.7%**	**0.666**	**0.928**
VGG-16	72.5%	0.659	0.912
MobileNetV2	70.6%	0.653	0.908
DenseNet-121	72.8%	0.652	0.926
ResNet-50	69.7%	0.642	0.913
ConvNeXt-Tiny	73.0%	0.639	0.917
EfficientNet-B0	68.9%	0.609	0.901
ViT-B/16 (Transformer)	64.0%	0.577	0.880



Honest verdict
We are **in the top group** (best F1 and AUC). But good CNNs are all within about ±3 points — with 960 test photos that is within chance. So: **'among the best, not far better'**. The Transformer (ViT) was worst — Transformers need much more data.

17c. Learning curve — does more data help?

Training data	Accuracy	AUC
25%	59.3%	0.850
50%	66.7%	0.882
75%	68.4%	0.898
100%	68.9%	0.901



More data helps, with smaller gains each step. AUC is still rising, so **more photos (the hospital dataset) should improve the model**, especially rare Glaucoma and Hypertension. This answers 'what if they used 100,000 images?' — more data generally helps, but papers with huge datasets often use a different task or split.

17d. Combining networks — ensemble search

Combination	Val F1	Test acc.	Test F1	RFMiD AUC
S13 + S12 (previous)	0.654	72.0%	0.655	0.800
S13 + S12 + DenseNet	0.660	72.6%	0.664	0.788
S13 + DenseNet (chosen)	**0.675**	**72.7%**	**0.666**	**0.771**
S13 + S12 + ConvNeXt	0.665	73.2%	0.664	0.796
All four	0.668	74.0%	0.676	0.787

- Tried 5 combinations and **chose on validation only**; a new one had to beat the old by at least 0.005 F1.
- The 4-network combination had a slightly higher **test** score (74.0%). We **did not pick it** — choosing after seeing test results is cheating. Say this in the viva; it shows discipline.
- Trade-off: DenseNet was trained on ODIR only, so RFMiD AUC fell 0.800 → 0.771.

17e. Batch tool recalibrated for the new model

New limits: Refer ≥ 30%, Normal ≥ 70%. Safer (4.3% missed, was 5.2%) but clears fewer healthy eyes (32.7%, was 38%). For screening, safer is better.

Things we chose NOT to do (they may ask)

We did not…	Because…
Use the leaking columns	They give fake high accuracy
Split by photo instead of by patient	It inflates the scores
Use the 124,000-photo IMAGES folder	Copies, plus a fake streak shortcut
Use SMOTE / oversampling	Creates artificial data; class weights are cleaner
Choose models by test score	Makes the test result meaningless
Use very large models	Too little data, limited GPU; the benchmark showed they don't win
Do fusion for 8 diseases	Age/sex add almost nothing there
Build multi-label (several diseases per photo)	Per-photo labels are single-disease; future improvement

Main limitations (say them before they do)

- Only age and sex as clinical data.
- Glaucoma and Hypertension are weak (few photos, subtle signs).
- The model drops on another hospital's photos — a third dataset is needed for a true test.
- The photo-quality check is rule-based and misses subtle bad photos.
- It is a research prototype, **not for clinical use**.

Next steps

- Use the hospital dataset.
- Retrain DenseNet on both datasets.
- Repeat the benchmark with several random seeds.
- Build a learned photo-quality model.
- Try a multi-label model.

Tech stack

Part	Tool
Language	Python 3.11
Deep learning (CNNs)	PyTorch + torchvision — MobileNetV2, EfficientNet-B0, DenseNet-121 (+ ResNet-50, VGG-16, ConvNeXt-Tiny, ViT for comparison)
Classical ML	scikit-learn — Logistic Regression, Random Forest, SVM, splits, scaling, metrics
Data	pandas, NumPy, Pillow
Charts	Matplotlib
Explainability	Grad-CAM (our own code with PyTorch hooks)
Web app	Streamlit
Hardware	RTX 4060 (8GB) laptop GPU, VS Code
Datasets	ODIR-5K (main), RFMiD (second hospital)

Summary — the whole project on one page

Step	What we did	Key result
Data	ODIR-5K: 6,392 photos, 3,358 patients, 8 conditions	Imbalanced; 15.7% multi-disease rows
Cleaning	Age=1 → missing; chose 'labels' column; removed leaking columns	Only age + sex are fair clinical inputs
Preprocessing	Patient-level stratified split; one-hot sex; standardised age (fit on train only)	Zero patient overlap
Clinical ML	LR, RF, SVM with class weights	RF 74.8% acc, F1 0.464 — age/sex too weak
Photo check	Validated folders; rejected duplicate IMAGES folder; colour fix	Used preprocessed_images (patient-linked)
Myopia CNN	MobileNetV2 transfer learning on GPU (PyTorch)	96.6% acc, F1 0.895
Fusion (myopia)	Photo features + age/sex, fine-tuned	97.4% acc, F1 0.875, recall 84%
8-disease CNN	MobileNetV2 → EfficientNet-B0 448px → ensemble	49.8% → 71.5% → 71.8% acc
Explainability	Grad-CAM heatmaps	Model looks at the right areas
Other hospital	Tested on RFMiD, then trained on both	AUC 0.752 → 0.800
Batch tool	Quality check + Refer / Review / Normal / Retake	94.6% of diseased eyes reach a doctor
Comparison	7 networks, same split; published papers; learning curve	Top group: best F1 & AUC
Final model	**EfficientNet-B0 (S13) + DenseNet-121 ensemble**	**72.7% acc, F1 0.666, AUC 0.928**
App	Streamlit: single photo, myopia, batch, about	Working demo

One-line answer for the guide
We built a leakage-free, patient-level pipeline: clinical ML baselines first, then CNN transfer learning on fundus photos, photo + clinical fusion for myopia, and an 8-disease model. We improved it step by step, explained it with Grad-CAM, tested it on a second hospital's data, added a batch screening tool, and fairly benchmarked it against 7 standard networks. Our final model is in the top group, with 72.7% accuracy and 0.928 AUC.

Three rules we followed everywhere
1. No leakage — no diagnosis columns as input, split by patient, fit preprocessing on training data only.
2. Choose on validation, test only once.
3. Report honestly — weak classes, drops on other hospitals and trade-offs are all written down.